UML-Diagramme – eine kurze Erläuterung

Im Folgenden wollen wir kurz betrachten, welche Aspekte für UML-Diagramme (Unified Modeling Language) wichtig sind. Dazu wollen wir die Entity-Klassen einer Kaffeemaschine betrachten, wie sie exemplarisch in Abb. 1 dargestellt sind.

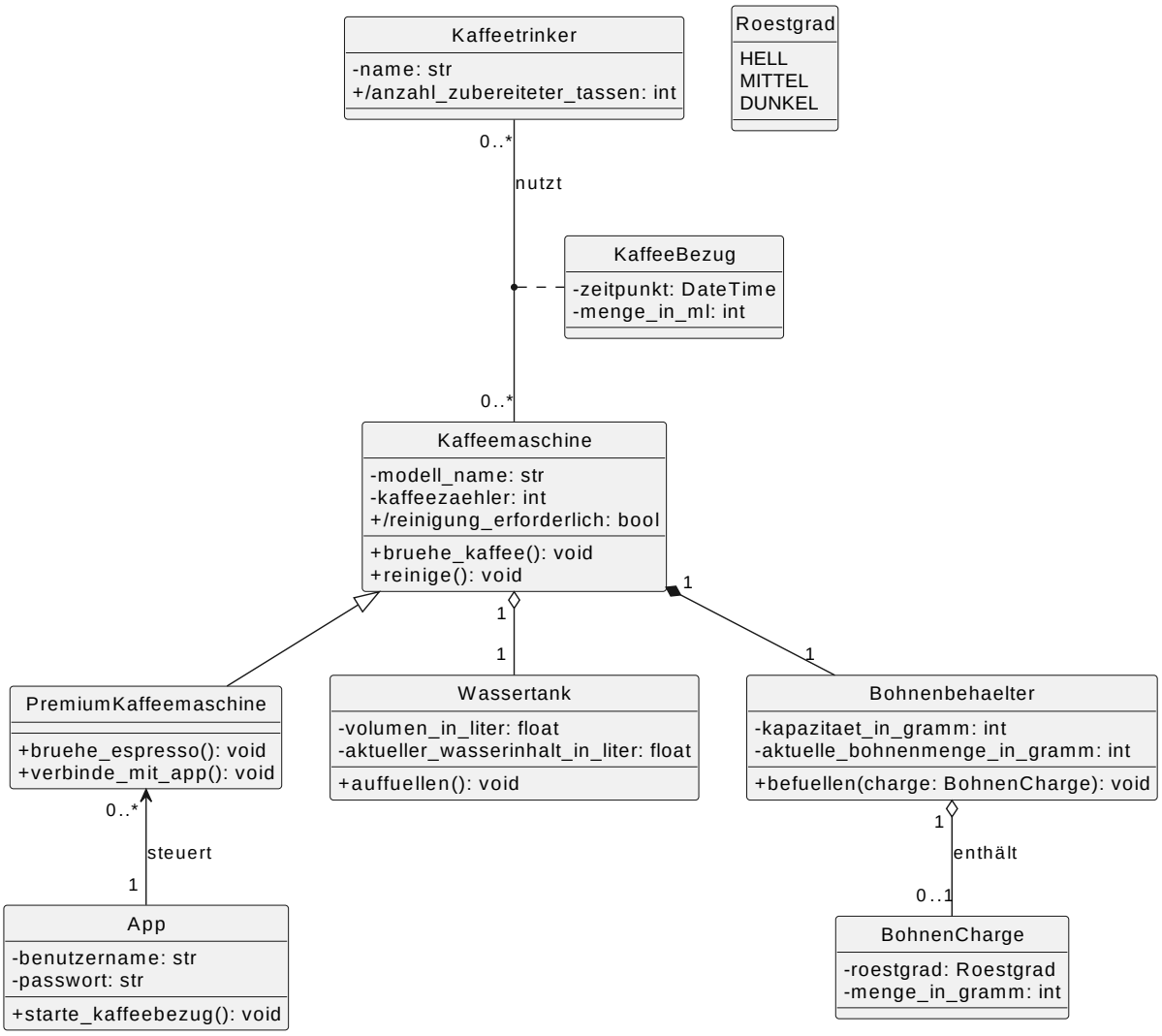


Abbildung 1: UML-Diagramm „Kaffeemaschine“

Was gehört ins UML-Klassendiagramm?

Ein UML-Klassendiagramm dient der Darstellung der statischen Struktur eines Systems. Es zeigt, welche Klassen existieren, wie sie miteinander in Beziehung stehen und welche Eigenschaften und Funktionen sie besitzen. Die zentrale Einheit im Klassendiagramm ist die Klasse selbst. Jede Klasse wird durch einen Namen identifiziert, der im Diagramm in der obersten Box der Klasse erscheint. Klassennamen werden üblicherweise in CamelCase geschrieben, beispielsweise `Kaffeemaschine` oder `PremiumKaffeemaschine`.
Zur Beschreibung einer Klasse gehören ihre Attribute, also die Merkmale oder Zustände, die das Objekt beschreibt. Diese Attribute werden mit Typ und Sichtbarkeit angegeben. Die Sichtbarkeit wird in UML durch Symbole dargestellt: `-` steht für nicht öffentlich gedachte Attribute und `+` für öffentliche Schnittstellen. Ein Attribut wie `-kaffeezaehler: int` beschreibt etwa einen internen Zähler, der die Anzahl der gebrühten Kaffees erfasst.

Attribute können außerdem als abgeleitete Größen dargestellt werden. In UML wird dies durch einen vorangestellten Schrägstrich gekennzeichnet. `+/reinigung_erforderlich: bool` oder `+/anzahl_zubereiteter_tassen: int` würden als Attribute beispielsweise Eigenschaften beschreiben, die sich aus anderen Zuständen oder Regeln ergeben und nicht zwingend direkt gespeichert werden müssen.
Darüber hinaus können UML-Klassendiagramme auch Enumerationstypen enthalten. Enumerationen beschreiben eine feste Menge erlaubter Werte und eignen sich insbesondere für fachlich eingeschränkte Zustände. Im Beispiel könnte ein Enumerationstyp `Roestgrad` die Werte `HELL`, `MITTEL` und `DUNKEL` enthalten.
Zusätzlich zu den Attributen können auch fachlich relevante Methoden dargestellt werden. In frühen Entity-Klassendiagrammen werden Methoden jedoch häufig nur sparsam verwendet, da der Schwerpunkt zunächst auf den fachlichen Klassen und deren Beziehungen liegt. Falls Methoden aufgenommen werden, sollten sie das objektorientierte Modell inhaltlich wesentlich ergänzen. Fachlich benannte Methoden wie `+bruehe_kaffee(): void` sind dabei sinnvoll. Reine technische Zugriffsmethoden wie Getter und Setter (z. B. `+get_kaffeezaehler(): int` oder `+set_kaffeezaehler(wert: int): void`) gehören dagegen in der Regel nicht ins UML-Klassendiagramm.
Konstruktoren, also spezielle Methoden zur Objekterzeugung, werden nur dann aufgenommen, wenn sie beim Erzeugen eines Objekts fachlich relevante Logik enthalten, zum Beispiel eine automatische Initialisierung mit bestimmten Werten.
Zusammenfassend gehört in ein UML-Klassendiagramm:

- der Klassenname,
- eine Liste der fachlich relevanten Attribute mit Sichtbarkeit und Typ,
- eine Auswahl fachlich relevanter Methoden,
- und ggf. ein Konstruktor mit besonderer fachlicher Bedeutung.

Beziehungen

Neben den einzelnen Klassen spielen insbesondere die Beziehungen zwischen den Klassen eine zentrale Rolle in UML-Klassendiagrammen. Sie beschreiben, wie Objekte miteinander verbunden sind und welche fachlichen Abhängigkeiten zwischen ihnen bestehen.

Assoziation

Eine Assoziation beschreibt eine lose Beziehung zwischen zwei Klassen, bei der ein Objekt ein anderes kennt oder nutzt. Die Lebensdauer der beteiligten Objekte ist dabei unabhängig. Im o. g. Beispiel ist es eine `App`, die eine `PremiumKaffeemaschine` steuert: Die App kennt die Kaffeemaschine und kann Methoden wie `bruehe_kaffee()` aufrufen, ohne selbst Teil der Kaffeemaschine zu sein.
Man unterscheidet zwischen gerichteter und ungerichteter Assoziation. Bei einer gerichteten Assoziation kennt nur eine Seite die andere. Im UML-Diagramm wird dies durch einen Pfeil ($\rightarrow$) dargestellt. Bei einer ungerichteten Assoziation kennen sich beide Klassen gegenseitig, dargestellt durch eine einfache Linie (—).
Assoziationen können mit Multiplizitäten versehen werden, zum Beispiel: eine App kann mehrere Kaffeemaschinen steuern (1 zu 0..*). Der Zweck einer Assoziation besteht in der Modellierung von Zusammenarbeit oder Referenzbeziehungen zwischen Objekten.

Assoziationsklasse

Eine Assoziationsklasse wird verwendet, wenn nicht nur die beteiligten Klassen selbst, sondern auch die Beziehung zwischen ihnen eigene Eigenschaften besitzt. In diesem Fall reicht eine einfache Assoziation nicht mehr aus, da zusätzliche Informationen direkt zur Beziehung gespeichert werden müssen.
Im Kaffeemaschinen-Beispiel beschreibt die Klasse `KaffeeBezug` die Beziehung zwischen einem `Kaffeetrinker` und einer `Kaffeemaschine`. Ein einzelner Kaffeebezug besitzt eigene fachliche Eigenschaften wie den Zeitpunkt oder die Getränkemenge. Die Beziehung selbst wird dadurch zu einem eigenständigen Objekt der Modellierung.
Ein `Kaffeetrinker` kann mehrere Kaffeebezüge starten, und eine `Kaffeemaschine` kann viele Kaffeebezüge ausführen. Jeder einzelne `KaffeeBezug` verbindet dabei genau einen `Kaffeetrinker` mit genau einer `Kaffeemaschine`.
Assoziationsklassen eignen sich insbesondere dann, wenn Beziehungen zusätzliche fachliche Informationen oder Zustände besitzen und daher selbst als eigenständige Konzepte der Anwendungsdomäne betrachtet werden müssen.

Aggregation

Eine Aggregation beschreibt eine „hat“-Beziehung zwischen Objekten, bei der das Teil unabhängig vom Ganzen existieren kann. Man spricht auch von einer losen Teil-Ganzes-Beziehung. Das bedeutet, dass das Teilobjekt nicht zwingend vom Lebenszyklus des Ganzen abhängig ist: wird das Ganze zerstört, kann das Teil weiterbestehen.
Ein Beispiel aus dem Kaffeemaschinen-Modell ist die Beziehung zwischen dem `Bohnenbehaelter` und einer `BohnenCharge`. Der Behälter enthält eine Bohnencharge, ist aber nicht deren alleiniger Besitzer. Die Bohnencharge könnte auch unabhängig existieren oder in einem anderen Behälter verwendet werden. Die Aggregation bringt zum Ausdruck, dass eine strukturelle Verbindung besteht, ohne dass die Bohnencharge exklusiv an einen bestimmten Behälter gebunden ist.
In UML-Diagrammen wird eine Aggregation durch eine Linie mit einer weißen Raute (◇—) am Ende des „Besitzers“ dargestellt. Wie bei Assoziationen können auch Aggregationen Multiplizitäten angeben, um zum Beispiel auszudrücken, dass ein Objekt mehrere andere enthalten kann. Die Aggregation dient der Modellierung einer strukturellen Beziehung, ohne Besitz oder vollständige Abhängigkeit auszudrücken.

Komposition

Eine Komposition beschreibt eine besonders starke „besteht-aus“-Beziehung zwischen Objekten, bei der das Teilobjekt vom Lebenszyklus des Ganzen vollständig abhängig ist. Das Teil ist exklusiv an das Ganze gebunden: wenn das Ganze gelöscht wird, wird auch das Teil mit gelöscht. Man spricht daher von einer festen Teil-Ganzes-Beziehung mit Besitz.
Im Kaffeemaschinen-Modell ist der `Bohnenbehaelter` ein gutes Beispiel für eine Komposition. Er gehört fest zur `Kaffeemaschine` und ist konstruktiv so integriert, dass er nicht sinnvoll unabhängig davon existieren kann. Wird die Maschine außer Betrieb genommen oder entsorgt, verliert auch der `Bohnenbehaelter` seine Funktion.
In UML-Diagrammen wird eine Komposition durch eine Linie mit einer ausgefüllten Raute (◆—) am Ende des Ganzen dargestellt. Auch hier können Multiplizitäten verwendet werden, um etwa die Anzahl der Teilobjekte zu kennzeichnen. Die Komposition modelliert somit eine enge Kopplung mit gemeinsamer Lebensdauer und Besitzverhältnis.

Vererbung

Vererbung beschreibt eine „ist-ein(e)“-Beziehung zwischen Klassen, bei der eine spezialisierte Klasse Eigenschaften und Verhalten einer allgemeineren Klasse übernimmt. Dadurch las-

sen sich gemeinsame Strukturen und Methoden in einer Oberklasse definieren, die von mehreren Unterklassen wiederverwendet werden können. Vererbung dient damit der Abstraktion und Wiederverwendbarkeit in objektorientierten Modellen. Im oben genannten Beispiel ist die `PremiumKaffeemaschine` ein Spezialfall der `Kaffeemaschine`. Sie erbt grundlegende Eigenschaften und Verhalten wie das Attribut `kaffeezaehler` oder die Methode `bruehe_kaffee()`. Zusätzlich erweitert sie die Oberklasse um weitere Funktionalitäten, hier insbesondere durch `bruehe_espresso()` und `verbinde_mit_app()`. Die `PremiumKaffeemaschine` ist also eine „besondere Art“ von `Kaffeemaschine`.
In UML-Diagrammen wird eine Vererbung durch einen Pfeil mit einem weißen, nicht ausgefüllten Dreieck (—▷) dargestellt. Die Pfeilspitze zeigt dabei immer zur Oberklasse. Vererbung wird verwendet, um Generalisierungen und Spezialisierungen zwischen Klassen auszudrücken.

Kardinalitäten (Multiplizitäten)

Kardinalitäten (auch Multiplizitäten genannt) geben an, wie viele Objekte einer Klasse mit einem Objekt einer anderen Klasse in Beziehung stehen können. Sie werden bei Assoziationen, Aggregationen und Kompositionen verwendet, um die Anzahl beteiligter Instanzen präzise zu modellieren. Übliche Angaben sind `1` (genau eins), `0..1` (optional), `0..*` (beliebig viele) oder konkrete Zahlenbereiche wie `1..3`.
Zum Beispiel bedeutet die Notation `1` ◇— `0..*`, dass genau ein Objekt der linken Klasse mit null oder mehr Objekten der rechten Klasse verbunden sein kann. Durch Kardinalitäten lassen sich wichtige fachliche Einschränkungen und Strukturregeln direkt im Diagramm visualisieren, was zu einem genaueren und nachvollziehbareren Modell beiträgt.

Rollennamen

Rollennamen beschreiben die Funktion oder Bedeutung, die ein Objekt einer Klasse im Kontext einer Beziehung zur anderen Klasse einnimmt. Sie stehen an einer Beziehungslinie und machen deutlich, wie Objekte zueinander in Beziehung stehen oder verwendet werden. Rollennamen tragen zur Lesbarkeit und Verständlichkeit eines Klassendiagramms bei, insbesondere bei komplexeren Modellen oder wechselseitigen Beziehungen.
In der Beziehung zwischen `Bohnenbehaelter` und `BohnenCharge` könnte beispielsweise der Rollenname `enthaelt` verwendet werden, um auszudrücken, dass der Behälter eine Bohnencharge enthält. Ebenso könnte bei der Beziehung zwischen `App` und `PremiumKaffeemaschine` der Rollenname `steuert` verwendet werden, um deutlich zu machen, dass die App die Maschine bedient. Rollennamen sind optional, aber besonders dann hilfreich, wenn die Beziehung nicht selbsterklärend ist, mehrere Beziehungen zwischen denselben Klassen bestehen oder eine bestimmte Sicht auf die Beziehung betont werden soll.

Darstellung von Beziehungen im UML-Diagramm

Die verschiedenen Beziehungstypen werden in UML durch charakteristische Symbole dargestellt. In textbasierten UML-Werkzeugen wie PlantUML existiert zusätzlich eine ASCII-Notation, mit der diese Beziehungen beschrieben werden können. Tabelle 1 zeigt die wichtigsten Beziehungstypen und ihre Darstellung in PlantUML.
Diese Symbole sind in UML-Tools wie PlantUML[1], Visual Paradigm[2] oder draw.io[3] standardisiert und werden dort automatisch grafisch korrekt umgesetzt. In textbasierten Tools lassen sich Beziehungen über ASCII-Notation beschreiben, was insbesondere in dokumentenorientierten oder kollaborativen Szenarien hilfreich ist.

[1] `https://plantuml.com/`
[2] `https://online.visual-paradigm.com/`
[3] `https://app.diagrams.net/`

Beziehung	Symbol	ASCII-Darstellung
Assoziation	A → B / A — B	`A --> B` (gerichtet) oder `A -- B` (ungerichtet)
Aggregation	A ◇— B	`A o-- B` (A aggregiert B)
Komposition	A ◆— B	`A *-- B` (A besitzt B vollständig)
Vererbung	A —▷ B	`A --|> B` (A erbt von B)

Tabelle 1: Darstellung von Beziehungen in UML-Klassendiagrammen

Entity-Klassen vs. vollständiges Klassendiagramm

Das bisher entwickelte UML-Klassendiagramm beschreibt die fachlich relevanten Objekte (Entitäten) der Anwendungsdomäne sowie deren Beziehungen untereinander. Die in Abb. 1 gezeigten Klassen bilden beispielsweise die Grundlage der Domäne „Kaffeemaschine“ und werden durch objektorientierte Beziehungen wie Vererbung, Assoziation, Aggregation oder Komposition miteinander verbunden.
Der Schwerpunkt liegt dabei zunächst auf den fachlichen Klassen, ihren Attributen und den Beziehungen zwischen den Objekten. Technische Aspekte der späteren Softwarearchitektur spielen in dieser frühen Modellierungsphase meist noch keine zentrale Rolle.
Für die spätere Umsetzung in Software wird das Entity-Modell anschließend um weitere technische Klassen und Architekturkomponenten erweitert. Eine ausführlichere Betrachtung der UML-Gesamtarchitektur erfolgt im Dokument zu Phase 2.

Was gehört nicht ins UML-Klassendiagramm?

Beim Erstellen eines UML-Klassendiagramms ist es wichtig, sich auf die fachlich relevanten Aspekte der Modellierung zu konzentrieren. Technische Details, die ausschließlich der konkreten Implementierung in einer bestimmten Programmiersprache dienen, sollten möglichst vermieden werden.
Dazu zählen insbesondere rein technische Methoden wie Getter und Setter (z. B. `get_name()` oder `set_kaffeezaehler(...)`), die keinen fachlichen Mehrwert bieten. Auch interne Hilfsmethoden zur Validierung, Initialisierung, Formatierung oder Zwischenschritten von Berechnungen gehören in der Regel nicht ins Klassendiagramm, da sie keine fachlichen Konzepte der Anwendungsdomäne beschreiben.
Ebenso sollten sprachspezifische Implementierungsdetails möglichst vermieden werden. Dazu gehören beispielsweise Python-Konventionen wie führende Unterstriche für interne Attribute (`_name`), konkrete Datenstrukturen wie `list[BohnenCharge]` oder Initialisierungen wie `kaffeezaehler = 0`. Das UML-Klassendiagramm soll die fachliche Struktur eines Systems beschreiben und nicht die konkrete Implementierung in einer bestimmten Programmiersprache vorwegnehmen.
Auch Kontrollstrukturen wie `if`, `for` oder `try` gehören nicht in ein UML-Klassendiagramm. Sie beschreiben Abläufe innerhalb von Methoden und werden, falls notwendig, in anderen Diagrammarten modelliert.

PlantUML-Code

PlantUML (`https://www.plantuml.com/`) ist ein freies Software-Dokumentationswerkzeug zur Erstellung von UML-Diagrammen unter der GNU General Public License. Es eignet sich hervorragend zur Erstellung komplexer Diagramme, worauf es sich nicht auf UML-Diagramme

beschränkt. Im Folgenden ist der Code gezeigt, der Abb. 1 erzeugt.

```
@startuml
hide circle
skinparam classAttributeIconSize 0
'skinparam linetype ortho 'für orthogonale Linien


' Basisklasse
class Kaffeemaschine {
  -modell_name: str
  -kaffeezaehler: int
  +/reinigung_erforderlich: bool
  +bruehe_kaffee(): void
  +reinige(): void
}

class PremiumKaffeemaschine {
  +bruehe_espresso(): void
  +verbinde_mit_app(): void
}

class App{
  -benutzername: str
  -passwort: str
'  +verbinde(): void
  +starte_kaffeebezug(): void
}

class Wassertank {
  -volumen_in_liter: float
  -aktueller_wasserinhalt_in_liter: float
  +auffuellen(): void
}

class BohnenCharge {
  -roestgrad: Roestgrad
  -menge_in_gramm: int
}

enum Roestgrad {
  HELL
  MITTEL
  DUNKEL
}

class Bohnenbehaelter {
  -kapazitaet_in_gramm: int
```

```
  -aktuelle_bohnenmenge_in_gramm: int
  +befuellen(charge: BohnenCharge): void
}


class Kaffeetrinker {
-name: str
+/anzahl_zubereiteter_tassen: int
}

class KaffeeBezug {
-zeitpunkt: DateTime
-menge_in_ml: int
}

' Aggregation: lose Verbindungen
Bohnenbehaelter "1 " o-- "0..1" BohnenCharge : enthält
Kaffeemaschine "1 " o-- "1 " Wassertank

' Komposition: fest verbaute Bestandteile
Kaffeemaschine " 1" *-- "1" Bohnenbehaelter

' Vererbung
PremiumKaffeemaschine -up-|> Kaffeemaschine

' Assoziation
App "1 " -up-> "0..* " PremiumKaffeemaschine : steuert
Kaffeetrinker "0..*" -- "0..*" Kaffeemaschine : nutzt

'Assoziationsklasse
(Kaffeetrinker, Kaffeemaschine) .. KaffeeBezug

@enduml
```

Abbildungsverzeichnis

1 UML-Diagramm „Kaffeemaschine“. Quelle: Walte (2026), eigene Darstellung . . . 1

Tabellenverzeichnis

1 Darstellung von Beziehungen in UML-Klassendiagrammen. Quelle: Walte (2025), eigene Darstellung 5